UADE



TP Integrador Obligatorio

INGENIERIA DE DATOS I

TURNO: NOCHE

GRUPO: Cafetear

Integrantes del Grupo:

- L 1247201 – Joaquín Morales Pessacq
- L 1174530 – Camila Pogonza
- L 1223910 – Guadalupe Olivera
- L 1131002 – Lautaro Patricio Rojas
- L 1205739 – Alexander Demmerle
- L 1224151 – Santiago Derito

Profesor:

Ing. Franco Emanuel Salazar

Fecha de Entrega:06/11/2026

Trabajo Practico
Integrador Obligatorio

Índice

1. Resumen Ejecutivo 3
2. Introducción 3
2.1. Contexto 3
2.2. Problemática 3
2.3. Solución propuesta 4
3. Objetivos del Sistema 4
4. Requisitos del Sistema 5
4.1. Requisitos Funcionales 5
4.2. Requisitos No Funcionales 5
5. Diseño de la Base de Datos 6
5.1. Modelo Conceptual 6
5.2. Modelo Lógico 7
5.3. Modelo Físico 7
5.4. Diccionario de Datos 8
5.5. Dependencias Funcionales 11
6. Implementación y Scripts SQL 12
6.1. Inserción de Datos 12
6.2. Consultas SQL 13
6.3. Triggers y Vistas 16
6.4. Funciones y Procedimientos 19
7. Interfaz de Usuario 21
8. Plan de Desarrollo y Metodología 23
9. Optimización y Rendimiento 23
10. Seguridad y Recuperación 24
10.1. Roles y permisos 24
10.2. Respaldo y recuperación 24
11. Pruebas y Validación 25
12. Gestión de Riesgos 26
13. Costos y Presupuesto 26
14. Conclusiones y Recomendaciones 27
15. Referencias y Anexos 28

Trabajo Practico
Integrador Obligatorio

1. Resumen Ejecutivo

Cafetear es una base de datos relacional en SQL Server que registra la calibración de la máquina de espresso de una cafetería de especialidad y la opinión de los clientes sobre cada café servido. Hoy ese conocimiento vive en la memoria de cada barista: cuando alguien se va, o cuando se abre una sucursal, se pierde lo que se había aprendido sobre cómo extraer cada lote.

El sistema guarda, para cada **calibración** (dosis, tiempo de extracción y rendimiento en taza), qué barista la hizo, con qué **lote** de café, y cuántas estrellas le puso cada **cliente**. Con eso se puede responder qué receta funciona mejor para cada grano, cómo evoluciona la calidad mes a mes, qué lotes decepcionan y qué baristas necesitan apoyo.

Resultados de esta entrega: 7 tablas con datos de prueba reproducibles (10 proveedores, 10 granos, 40 lotes, 10 baristas, 60 clientes, 225 calibraciones y 346 calificaciones), 3 procedimientos, 2 vistas, 1 función, 2 triggers, 3 índices, 2 roles de seguridad y una copia de respaldo verificada. Los datos de prueba son siempre los mismos, así que todos los integrantes trabajan con exactamente los mismos datos.

2. Introducción

2.1.Contexto

Una cafetería de especialidad ajusta su molino y su máquina cada día: la dosis de café molido, el tiempo de extracción y los gramos que caen en la taza cambian con cada lote, con la frescura del tostado y hasta con el clima. A ese ajuste se lo llama **calibración**. El negocio se complica porque los baristas rotan con frecuencia y se abren y cierran locales, de modo que la receta que funcionaba rara vez se transmite.

2.2.Problemática

- No hay registro histórico de qué dosis, tiempo y rendimiento se usaron con cada lote.
- La opinión del cliente no se vincula con la forma en que se preparó el café, así que no se sabe qué calibraciones agradan más.
- No se puede comparar el desempeño entre baristas ni detectar a tiempo que un lote nuevo está saliendo mal.
- Cuando un barista se va, se pierde su conocimiento; el reemplazo empieza de cero.

2.3.Solución propuesta

Una base de datos que relaciona **Proveedor → Lote → Calibración → Calificación**, con baristas y clientes como actores. Está pensada para mejorar los resultados y la calidad que recibe el cliente: la pregunta central es qué extracciones funcionan mejor y cómo se comportan los clientes al calificarlas.

Decisión de diseño: se descartaron las mezclas (blends) de granos para no complicar el modelo; cada lote corresponde a un único grano. Se agregó la entidad **Lote** porque la misma variedad cambia entre bolsas según el tostado y la frescura, y eso afecta la receta.

3. Objetivos del Sistema

Objetivo general: centralizar el conocimiento de calibración y la opinión del cliente para sostener y mejorar la calidad de la taza.

- Registrar cada calibración con sus tres variables (dosis, tiempo, rendimiento) y asociarla a un barista y a un lote.
- Registrar las calificaciones (1 a 5) de los clientes y vincularlas a la calibración que las originó.
- Identificar la receta de referencia de cada grano y la evolución mensual de la calidad.
- Detectar lotes problemáticos, baristas que necesitan capacitación y clientes con criterios opuestos.
- Garantizar integridad con triggers y claves, y separar permisos por perfil.
- Permitir que cualquier integrante levante el entorno con los mismos datos en minutos.

4. Requisitos del Sistema

4.1. Requisitos Funcionales

ID	Requisito	Cómo se resuelve
RF-01	Registrar una calibración con dosis, tiempo de extracción y rendimiento, asociada a un barista y a un lote	Procedimiento `uspCalibracionCrear`
RF-02	Registrar la calificación de un cliente sin que tenga que indicar la calibración	Procedimiento `uspCalificacionCrear` y trigger `trgCalificacionAsignarCalibracion`
RF-03	Calcular la relación de extracción (rendimiento ÷ dosis)	Función `fnRelacionExtraccion`
RF-04	Conocer el puntaje promedio de cada calibración y la mejor receta de cada grano	Vistas `vwPuntajePorCalibracion` y `vwMejorCalibracionPorGrano`
RF-05	Ranking de baristas por período	Procedimiento `uspRankingBaristas`
RF-06	Rechazar datos inválidos (puntaje fuera de rango, calificación repetida o anterior a la calibración)	Trigger `trgCalificacionControl`

4.2. Requisitos No Funcionales

ID	Requisito	Cómo se cumple
RNF-01	Reproducibilidad: mismos datos para todo el equipo	Datos de prueba fijos: se verificó con `CHECKSUM_AGG` que dos cargas desde cero dan datos idénticos
RNF-02	Integridad	Claves primarias, foráneas, `UNIQUE` en CUIT y DNI, y triggers de negocio
RNF-03	Rendimiento	3 índices sobre claves foráneas y fecha
RNF-04	Seguridad	2 roles; el barista no tiene permisos sobre las tablas, solo sobre procedimientos y una vista
RNF-05	Recuperabilidad	Backup completo con `CHECKSUM` y verificación con `RESTORE VERIFYONLY`
RNF-06	Mantenibilidad	Objetos que se pueden volver a crear sin errores (`CREATE OR ALTER`) y nombres consistentes en camelCase

5. Diseño de la Base de Datos

5.1.Modelo Conceptual

El modelo tiene siete entidades y seis relaciones uno-a-muchos. No hay relaciones muchos-a-muchos puras: la relación entre clientes y calibraciones se resuelve con la entidad **Calificación**, que además aporta el atributo propio (puntaje) y la fecha.

- Un **Proveedor** provee muchos **Lotes**; cada lote lo provee un solo proveedor.
- Un **Grano** se compra en muchos **Lotes**; cada lote es de un único grano.
- Un **Lote** se usa en muchas **Calibraciones**.
- Un **Barista** realiza muchas **Calibraciones**.
- Una **Calibración** recibe muchas **Calificaciones**. El cliente no elige la calibración: el sistema le asigna la más próxima anterior a la fecha de su calificación.
- Un **Cliente** deja muchas **Calificaciones**.

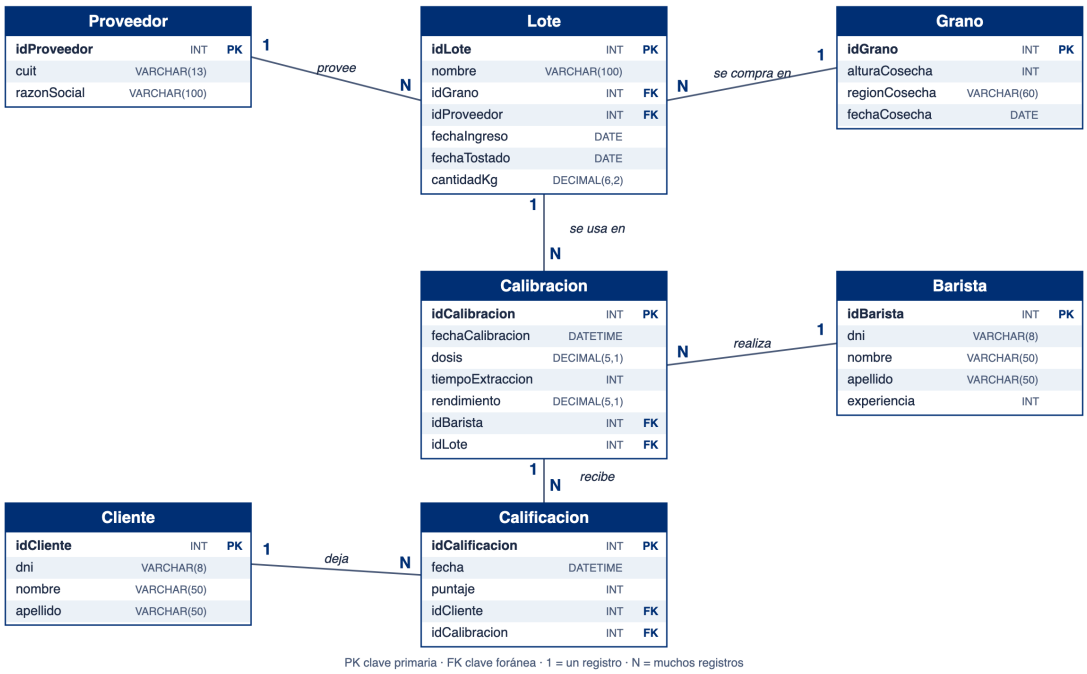


Figura 1. Diagrama entidad-relación de Cafetear

5.2.Modelo Lógico

Pasaje a tablas (clave primaria **PK**, foránea **FK**):

Tabla	Atributos
Proveedor	**idProveedor** (PK), cuit, razonSocial
Grano	**idGrano** (PK), alturaCosecha, regionCosecha, fechaCosecha
Lote	**idLote** (PK), nombre, idGrano (FK), idProveedor (FK), fechaIngreso, fechaTostado, cantidadKg
Barista	**idBarista** (PK), dni, nombre, apellido, experiencia
Cliente	**idCliente** (PK), dni, nombre, apellido
Calibracion	**idCalibracion** (PK), fechaCalibracion, dosis, tiempoExtraccion, rendimiento, idBarista (FK), idLote (FK)
Calificacion	**idCalificacion** (PK), fecha, puntaje, idCliente (FK), idCalibracion (FK)

5.3.Modelo Físico

Definición de las tablas en T-SQL (SQL Server 2022). Las tablas con claves que carga el usuario (Grano, Lote, Calibracion y Calificacion) usan `INT PRIMARY KEY` y el siguiente ID lo calculan los procedimientos; Proveedor, Barista y Cliente usan `IDENTITY`.

```
CREATE TABLE Proveedor (
    idProveedor INT IDENTITY(1,1) PRIMARY KEY,
    cuit        VARCHAR(13)  NOT NULL UNIQUE,
    razonSocial VARCHAR(100) NOT NULL
);

CREATE TABLE Grano (
    idGrano       INT PRIMARY KEY,
    alturaCosecha INT          NOT NULL,   -- metros sobre el nivel del mar
    regionCosecha VARCHAR(60)  NOT NULL,
    fechaCosecha  DATE         NOT NULL
);

CREATE TABLE Lote (
    idLote       INT PRIMARY KEY,
    nombre       VARCHAR(100) NOT NULL,   -- nombre de la finca, microlote o proceso
    idGrano      INT NOT NULL REFERENCES Grano(idGrano),
    idProveedor  INT NOT NULL REFERENCES Proveedor(idProveedor),
    fechaIngreso DATE NOT NULL,
    fechaTostado DATE NOT NULL,
    cantidadKg   DECIMAL(6,2) NOT NULL
);
```

```
CREATE TABLE Barista (
    idBarista   INT IDENTITY(1,1) PRIMARY KEY,
    dni         VARCHAR(8)  NOT NULL UNIQUE,
    nombre      VARCHAR(50) NOT NULL,
    apellido    VARCHAR(50) NOT NULL,
    experiencia INT         NOT NULL       -- años de experiencia
);

CREATE TABLE Cliente (
    idCliente INT IDENTITY(1,1) PRIMARY KEY,
    dni       VARCHAR(8)  NOT NULL UNIQUE,
    nombre    VARCHAR(50) NOT NULL,
    apellido  VARCHAR(50) NOT NULL
);

CREATE TABLE Calibracion (
    idCalibracion    INT PRIMARY KEY,
    fechaCalibracion DATETIME     NOT NULL,
    dosis            DECIMAL(5,1) NOT NULL,   -- gramos de café molido
    tiempoExtraccion INT          NOT NULL,   -- segundos
    rendimiento      DECIMAL(5,1) NOT NULL,   -- gramos de café en la taza
    idBarista        INT NOT NULL REFERENCES Barista(idBarista),
    idLote           INT NOT NULL REFERENCES Lote(idLote)
);

CREATE TABLE Calificacion (
    idCalificacion INT PRIMARY KEY,
    fecha          DATETIME NOT NULL,
    puntaje        INT      NOT NULL,
    idCliente      INT NOT NULL REFERENCES Cliente(idCliente),
    idCalibracion  INT NOT NULL REFERENCES Calibracion(idCalibracion)
);
```

5.4.Diccionario de Datos

Tabla Proveedor

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
idProveedor	4	INT	Identificador (IDENTITY)	No	PK	Autoincremental
cuit	13	VARCHAR	CUIT del proveedor	No	-	Único
razonSocial	100	VARCHAR	Nombre de la empresa	No	-	-

Tabla Grano

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
idGrano	4	INT	Identificador del grano	No	PK	-
alturaCosecha	4	INT	Metros sobre el nivel del mar	No	-	-
regionCosecha	60	VARCHAR	Región de origen	No	-	-
fechaCosecha	3	DATE	Fecha de cosecha	No	-	-

Tabla Lote

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
idLote	4	INT	Identificador del lote	No	PK	-
nombre	100	VARCHAR	Finca, microlote o proceso	No	-	-
idGrano	4	INT	Grano del lote	No	FK → Grano	-
idProveedor	4	INT	Quien lo vendió	No	FK → Proveedor	-
fechaIngreso	3	DATE	Cuándo llegó al local	No	-	Procedimiento: no calibrar antes
fechaTostado	3	DATE	Fecha de tostado	No	-	-
cantidadKg	5	DECIMAL(6, 2)	Kilos recibidos	No	-	-

Trabajo Practico
Integrador Obligatorio

Tabla Barista

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
idBarista	4	INT	Identificador (IDENTITY)	No	PK	Autoincremental
dni	8	VARCHAR	Documento	No	-	Único
nombre	50	VARCHAR	Nombre	No	-	-
apellido	50	VARCHAR	Apellido	No	-	-
experiencia	4	INT	Años de experiencia	No	-	-

Tabla Cliente

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
idCliente	4	INT	Identificador (IDENTITY)	No	PK	Autoincremental
dni	8	VARCHAR	Documento	No	-	Único
nombre	50	VARCHAR	Nombre	No	-	-
apellido	50	VARCHAR	Apellido	No	-	-

Tabla Calibracion

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
idCalibracion	4	INT	Identificador	No	PK	-
fechaCalibracion	8	DATETIME	Momento de la prueba	No		-
dosis	5	DECIMAL(5, 1)	Gramos de café molido	No	-	> 0 (procedimiento)
tiempoExtraccion	4	INT	Segundos de extracción	No	-	-
rendimiento	5	DECIMAL(5, 1)	Gramos de café en la taza	No	-	-
idBarista	4	INT	Quién calibró	No	FK → Barista	-
idLote	4	INT	Lote usado	No	FK → Lote	-

Tabla Calificacion

Campo	Tamaño (bytes)	Tipo	Descripción	NULL	Clave	Restricción
idCalificacion	4	INT	Identificador	No	PK	-
fecha	8	DATETIME	Cuándo calificó	No	-	Trigger: ≥ calibración
puntaje	4	INT	Estrellas de 1 a 5	No	-	Trigger: 1 a 5
idCliente	4	INT	Quién calificó	No	FK → Cliente	Trigger: una por calibración
idCalibracion	4	INT	Calibración calificada	No	FK → Calibracion	-

5.5.Dependencias Funcionales

Todas las tablas están en **Tercera Forma Normal (3FN)**: cada atributo no clave depende de la clave completa y solo de ella. Las dependencias principales son:

- idProveedor → cuit, razonSocial (y cuit → idProveedor, por ser clave candidata).
- idGrano → alturaCosecha, regionCosecha, fechaCosecha.

- idLote → nombre, idGrano, idProveedor, fechaIngreso, fechaTostado, cantidadKg.
- idBarista → dni, nombre, apellido, experiencia. idCliente → dni, nombre, apellido.
- idCalibracion → fechaCalibracion, dosis, tiempoExtraccion, rendimiento, idBarista, idLote.
- idCalificacion → fecha, puntaje, idCliente, idCalibracion.

La región y la altura dependen del grano, no del lote; por eso viven en **Grano** y no se repiten en cada lote. La relación de extracción y el puntaje promedio son datos derivados y no se almacenan: se calculan con una función y vistas, para evitar redundancia.

6. Implementación y Scripts SQL

6.1. Inserción de Datos

Los datos de prueba se generaron con una semilla fija: son siempre los mismos, así que todo el equipo trabaja con exactamente los mismos datos. Las calificaciones no son aleatorias puras: la calidad sube con la curva de aprendizaje, cae en meses con eventos (cambio de molino, ola de calor) y hay lotes buenos, malos y en declive, para que las consultas tengan algo interesante que mostrar.

Filas cargadas (consulta real sobre la base):

tabla	filas
Proveedor	10
Grano	10
Lote	40
Barista	10
Cliente	60
Calibracion	225
Calificacion	346

Ejemplo de carga de maestros:

```
USE cafetear;
GO
SET NOCOUNT ON;
```

```
-- Proveedor (los CUIT son inventados)

INSERT INTO Proveedor (cuit, razonSocial) VALUES
    ('30-71458230-4', 'Café Andino S.R.L.'),
    ('30-70912345-1', 'Tostadores del Sur S.A.'),
    ('30-71203344-7', 'Finca Los Altos S.A.'),
```

6.2.Consultas SQL

1. Lotes con puntaje promedio menor a 3,5

```
SELECT l.nombre,
       COUNT(c.idCalibracion) AS cantidadCalibraciones,
       SUM(c.dosis) AS gramosTotalesUsados,
       SUM(p.sumaPuntajes * 1.0) / SUM(p.cantidadCalificaciones) AS puntajePromedio
FROM Lote l
JOIN Calibracion c ON c.idLote = l.idLote
LEFT JOIN (SELECT idCalibracion,
                  SUM(puntaje) AS sumaPuntajes,
                  COUNT(*) AS cantidadCalificaciones
           FROM Calificacion
           GROUP BY idCalibracion) p ON p.idCalibracion = c.idCalibracion
GROUP BY l.idLote, l.nombre
HAVING SUM(p.sumaPuntajes * 1.0) / SUM(p.cantidadCalificaciones) < 3.5;
```

2. Puntaje promedio y cantidad de calificaciones de cada mes.

```
SELECT CONCAT(MONTH(fecha), '/', YEAR(fecha)) AS mesAnio,
       AVG(puntaje) * 1.0 AS calificacionPromedio,
       COUNT(*) AS cantidadCalificaciones
FROM Calificacion
GROUP BY MONTH(fecha), YEAR(fecha)
ORDER BY MONTH(fecha), YEAR(fecha);
```

3. Puntaje promedio según las semanas transcurridas desde el tostado, por grano y proveedor

```
SELECT g.idGrano,
       g.regionCosecha AS grano,
```

```
       p.razonSocial AS proveedor,
       DATEDIFF(DAY, l.fechaTostado, c.fechaCalibracion) / 7 AS semanasDesdeTostado,
       AVG(cf.puntaje * 1.0) AS puntajePromedio,
       COUNT(*) AS cantidadCalificaciones
FROM Grano g
JOIN Lote l ON l.idGrano = g.idGrano
JOIN Proveedor p ON p.idProveedor = l.idProveedor
JOIN Calibracion c ON c.idLote = l.idLote
JOIN Calificacion cf ON cf.idCalibracion = c.idCalibracion
GROUP BY g.idGrano,
         g.regionCosecha,
         p.idProveedor,
         p.razonSocial,
         DATEDIFF(DAY, l.fechaTostado, c.fechaCalibracion) / 7
ORDER BY g.idGrano, p.razonSocial, semanasDesdeTostado;
```

4. Para cada grano y proveedor, la calibración con mayor calificación promedio

```
SELECT g.regionCosecha AS grano,
       p.razonSocial AS proveedor,
       v.idCalibracion,
       v.dosis,
       v.tiempoExtraccion,
       v.rendimiento,
       v.puntajePromedio,
       v.calificaciones
FROM vwPuntajePorCalibracion v
JOIN Lote l ON l.idLote = v.idLote
JOIN Grano g ON g.idGrano = l.idGrano
JOIN Proveedor p ON p.idProveedor = l.idProveedor
WHERE v.idCalibracion = (SELECT TOP 1 v2.idCalibracion
                         FROM vwPuntajePorCalibracion v2
                         JOIN Lote l2 ON l2.idLote = v2.idLote
                         WHERE l2.idGrano = l.idGrano
                           AND l2.idProveedor = l.idProveedor   -- mismo grano y
mismo proveedor
                           AND v2.calificaciones >= 3
                         ORDER BY v2.puntajePromedio DESC, v2.calificaciones DESC,
v2.idCalibracion)
ORDER BY g.regionCosecha, p.razonSocial;
```

5. Proveedor con mejor café (con más de 5 calificaciones)

```
SELECT TOP 1 p.idProveedor,
             p.razonSocial,
             ROUND(AVG(cf.puntaje * 1.0), 2) AS puntajePromedio
FROM Proveedor p
JOIN Lote l ON l.idProveedor = p.idProveedor
JOIN Calibracion cl ON cl.idLote = l.idLote
JOIN Calificacion cf ON cf.idCalibracion = cl.idCalibracion
GROUP BY p.idProveedor, p.razonSocial
HAVING COUNT(cf.idCalificacion) > 5
```

```
ORDER BY AVG(cf.puntaje * 1.0) DESC;
```

6. Clientes que calificaron alguna vez y no calificaron en los últimos 90 días

```
SELECT CONCAT(c.nombre, ' ', c.apellido) AS nombreCliente,
       c.dni
FROM Cliente c
JOIN Calificacion cal ON cal.idCliente = c.idCliente
GROUP BY c.idCliente, c.nombre, c.apellido, c.dni
HAVING DATEDIFF(DAY, MAX(cal.fecha), GETDATE()) > 90;
```

7. Calidad por día de la semana

```
SELECT DATENAME(WEEKDAY, fecha) AS diaDeLaSemana,
       AVG(puntaje * 1.0) AS calificacionPromedio
FROM Calificacion
GROUP BY DATENAME(WEEKDAY, fecha), DATEPART(WEEKDAY, fecha)
ORDER BY DATEPART(WEEKDAY, fecha);
```

8. Lotes cuya calificación promedio del último mes es menor a la del mes anterior

```
SELECT l.idLote,
       l.nombre AS lote,
       AVG(cf.puntaje * 1.0) AS promedioUltimoMes,
       (SELECT AVG(cf2.puntaje * 1.0)
        FROM Calibracion c2
        JOIN Calificacion cf2 ON cf2.idCalibracion = c2.idCalibracion
        WHERE c2.idLote = l.idLote
          AND cf2.fecha >= DATEADD(MONTH, DATEDIFF(MONTH, 0, GETDATE()) - 2, 0)
          AND cf2.fecha <  DATEADD(MONTH, DATEDIFF(MONTH, 0, GETDATE()) - 1, 0)) AS
promedioMesAnterior,
       COUNT(*) AS calificacionesUltimoMes
FROM Lote l
JOIN Calibracion c ON c.idLote = l.idLote
JOIN Calificacion cf ON cf.idCalibracion = c.idCalibracion
WHERE cf.fecha >= DATEADD(MONTH, DATEDIFF(MONTH, 0, GETDATE()) - 1, 0)
  AND cf.fecha <  DATEADD(MONTH, DATEDIFF(MONTH, 0, GETDATE()), 0)
GROUP BY l.idLote, l.nombre
HAVING AVG(cf.puntaje * 1.0) < (SELECT AVG(cf2.puntaje * 1.0)
                                FROM Calibracion c2
                                JOIN Calificacion cf2 ON cf2.idCalibracion =
c2.idCalibracion
                                WHERE c2.idLote = l.idLote
                                  AND cf2.fecha >= DATEADD(MONTH, DATEDIFF(MONTH, 0,
GETDATE()) - 2, 0)
                                  AND cf2.fecha <  DATEADD(MONTH, DATEDIFF(MONTH, 0,
GETDATE()) - 1, 0))
ORDER BY promedioUltimoMes;
```

9. Diferencia entre la mejor y la peor calificación de cada lote

```
SELECT l.nombre AS lote,
       p.razonSocial AS proveedor,
       MIN(cf.puntaje) AS peorPuntaje,
       MAX(cf.puntaje) AS mejorPuntaje,
       MAX(cf.puntaje) - MIN(cf.puntaje) AS diferencia,
       AVG(cf.puntaje * 1.0) AS puntajePromedio,
       COUNT(*) AS cantidadCalificaciones
FROM Lote l
JOIN Proveedor p ON p.idProveedor = l.idProveedor
JOIN Calibracion c ON c.idLote = l.idLote
JOIN Calificacion cf ON cf.idCalibracion = c.idCalibracion
GROUP BY l.idLote, l.nombre, p.razonSocial
HAVING COUNT(*) >= 3
ORDER BY diferencia DESC;
```

10. Lotes cuya calificación promedio del último mes es menor a la del mes anterior

```
SELECT TOP 5 c.tiempoExtraccion AS tiempoSeg,
             AVG(cf.puntaje * 1.0) AS puntajePromedio,
             COUNT(*) AS cantidadCalificaciones,
             COUNT(DISTINCT c.idCalibracion) AS cantidadCalibraciones
FROM Calibracion c
JOIN Calificacion cf ON cf.idCalibracion = c.idCalibracion
GROUP BY c.tiempoExtraccion
HAVING COUNT(*) >= 5
ORDER BY puntajePromedio DESC, cantidadCalificaciones DESC;
```

6.3.Triggers y Vistas

Triggers. Aplican las reglas de negocio dentro de la base: valen aunque alguien inserte directamente, sin pasar por un procedimiento. Ante una violación hacen `ROLLBACK` y lanzan un error con `THROW`.

```
-- Control de calificaciones: puntaje de 1 a 5, no anterior a la calibración y una
sola por cliente y calibración.
CREATE OR ALTER TRIGGER dbo.trgCalificacionControl
ON dbo.Calificacion
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1 FROM inserted WHERE puntaje NOT BETWEEN 1 AND 5)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50020, N'El puntaje debe estar entre 1 y 5.', 1;
    END;

    IF EXISTS (SELECT 1
```

```
                FROM inserted i
                JOIN Calibracion c ON c.idCalibracion = i.idCalibracion
                WHERE i.fecha < c.fechaCalibracion)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50021, N'La calificación no puede ser anterior a la calibración.', 1;
    END;

    IF EXISTS (SELECT 1
                FROM Calificacion k
                JOIN inserted i ON i.idCliente = k.idCliente AND i.idCalibracion =
k.idCalibracion
                GROUP BY k.idCliente, k.idCalibracion
                HAVING COUNT(*) > 1)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50022, N'Ese cliente ya calificó esta calibración.', 1;
    END;
END
GO
```

El segundo trigger es `INSTEAD OF INSERT`: como el cliente no sabe qué calibración le tocó, intercepta el insert de una calificación y le asigna la calibración más próxima anterior a su fecha. Vale para cualquier insert, venga o no de un procedimiento; si no hay ninguna calibración anterior, lo rechaza.

```
-- Asignación de calibración: el cliente no sabe qué calibración le tocó, así que se
le asigna la más próxima anterior a su calificación.
-- Vale para cualquier INSERT, aunque no pase por un procedimiento; si el insert
trae un idCalibracion, se reemplaza por el que corresponde.
CREATE OR ALTER TRIGGER dbo.trgCalificacionAsignarCalibracion
ON dbo.Calificacion
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1
                FROM inserted i
                WHERE NOT EXISTS (SELECT 1 FROM Calibracion c WHERE
c.fechaCalibracion <= i.fecha))
        THROW 50023, N'No hay ninguna calibración anterior a esa fecha.', 1;

    INSERT INTO Calificacion (idCalificacion, fecha, puntaje, idCliente,
idCalibracion)
    SELECT i.idCalificacion, i.fecha, i.puntaje, i.idCliente, nearest.idCalibracion
    FROM inserted i
    CROSS APPLY (SELECT TOP 1 c.idCalibracion
                 FROM Calibracion c
                 WHERE c.fechaCalibracion <= i.fecha
                 ORDER BY c.fechaCalibracion DESC, c.idCalibracion DESC) nearest;
```

```
END
GO
```

Vistas. Se crearon dos. `vwPuntajePorCalibracion` usa la función `fnRelacionExtraccion` y la reutilizan `vwMejorCalibracionPorGrano` y el procedimiento `uspRankingBaristas`, para no repetir los `JOIN` ni los cálculos; `vwMejorCalibracionPorGrano` usa `NOT EXISTS` para quedarse con la mejor calibración de cada grano (con al menos 3 calificaciones, para que el promedio sea confiable).

```
-- La receta de referencia de cada grano: la calibración con mejor puntaje promedio
(con al menos 3 calificaciones).
CREATE OR ALTER VIEW dbo.vwMejorCalibracionPorGrano AS
SELECT g.idGrano, g.regionCosecha, v.idCalibracion, v.fechaCalibracion, v.dosis,
v.tiempoExtraccion,
       v.rendimiento, v.relacionExtraccion, v.calificaciones, v.puntajePromedio
FROM vwPuntajePorCalibracion v
JOIN Grano g ON g.idGrano = v.idGrano
WHERE v.calificaciones >= 3
  AND NOT EXISTS (SELECT 1
                  FROM vwPuntajePorCalibracion otra
                  WHERE otra.idGrano = v.idGrano
                    AND otra.calificaciones >= 3
                    AND otra.puntajePromedio > v.puntajePromedio);
GO
```

Resultado de la vista `vwMejorCalibracionPorGrano`:

idGrano	regionCosecha	idCalibracion	dosis	tiempoExtraccion	rendimiento	relacion	calif	puntaje
1	Huila, Colombia	145	20.3	31	42.1	2.07	4	5
2	Cerrado Mineiro, Brasil	200	20.5	28	42.2	2.06	4	4.5
3	Yirgacheffe, Etiopía	220	18.8	30	35.7	1.9	3	5
3	Yirgacheffe, Etiopía	208	18.2	29	35	1.92	4	5
4	Antigua, Guatemala	153	18.3	26	39.3	2.15	7	4.57
5	Sul de Minas, Brasil	114	17	30	35.8	2.11	6	4.33
6	Nariño, Colombia	135	19.1	29	40.3	2.11	6	4.83
7	Sidamo, Etiopía	108	19.6	31	41.8	2.13	4	4.25
7	Sidamo, Etiopía	138	17.1	28	34	1.99	4	4.25
8	Tarrazú, Costa Rica	218	18.2	30	34.7	1.91	8	4.13

6.4.Funciones y Procedimientos

Función: `fnRelacionExtraccion` calcula cuántos gramos de café salen en la taza por cada gramo molido. Es escalar y la usa la vista `vwPuntajePorCalibracion`.

```
-- Relación de extracción: gramos de café en la taza por cada gramo de café molido.
Con 18 g y 36 g da 2,00 (1:2).
CREATE OR ALTER FUNCTION dbo.fnRelacionExtraccion (@dosis DECIMAL(5,1), @rendimiento
DECIMAL(5,1))
RETURNS DECIMAL(5,2)
AS
BEGIN
    RETURN CASE WHEN @dosis > 0 THEN CAST(@rendimiento / @dosis AS DECIMAL(5,2))
END;
END
```

Procedimientos. Son tres y están escritos de forma simple. `uspCalibracionCrear` valida los datos con `THROW`, calcula el siguiente ID y lo devuelve:

```
-- Registrar una calibración: valida los datos, calcula el próximo ID y lo devuelve.
CREATE OR ALTER PROCEDURE uspCalibracionCrear
    @fechaCalibracion DATETIME,
    @dosis DECIMAL(5,1),
    @tiempoExtraccion INT,
    @rendimiento DECIMAL(5,1),
    @idBarista INT,
```

```
    @idLote INT
AS
BEGIN
    IF @dosis <= 0
        THROW 50000, N'La dosis debe ser mayor a cero.', 1;
    IF @tiempoExtraccion <= 0
        THROW 50000, N'El tiempo de extracción debe ser mayor a cero.', 1;
    IF @rendimiento <= 0
        THROW 50000, N'El rendimiento debe ser mayor a cero.', 1;
    IF EXISTS (SELECT 1 FROM Lote WHERE idLote = @idLote AND CAST(@fechaCalibracion
AS DATE) < fechaIngreso)
        THROW 50000, N'La calibración no puede ser anterior al ingreso del lote.',
1;

    DECLARE @idCalibracion INT = (SELECT ISNULL(MAX(idCalibracion), 0) + 1 FROM
Calibracion);

    INSERT INTO Calibracion (idCalibracion, fechaCalibracion, dosis,
tiempoExtraccion, rendimiento, idBarista, idLote)
    VALUES (@idCalibracion, @fechaCalibracion, @dosis, @tiempoExtraccion,
@rendimiento, @idBarista, @idLote);

    SELECT @idCalibracion AS idCalibracion;
END
GO
```

`uspCalificacionCrear` no recibe `idCalibracion`: la asigna el trigger `trgCalificacionAsignarCalibracion`. El tercero, `uspRankingBaristas`, tiene período opcional y se apoya en la vista `vwPuntajePorCalibracion`:

```
EXEC uspRankingBaristas @desde = '2026-01-01', @hasta = '2026-10-31';
```

idBarista	Barista	calibraciones	calificaciones	puntajePromedio
6	Julián Acosta	5	6	4.33
2	Lucas Benítez	22	41	4.24
5	Valentina Pereyra	14	28	4.04
1	Martina Ríos	33	68	3.9
7	Camila Ferrer	9	20	3.65
9	Florencia Montero	9	11	3.45
10	Tomás Arias	8	14	3.43
3	Sofía Carrizo	18	35	2.97
4	Mateo Duarte	9	14	2.36

7. Interfaz de Usuario

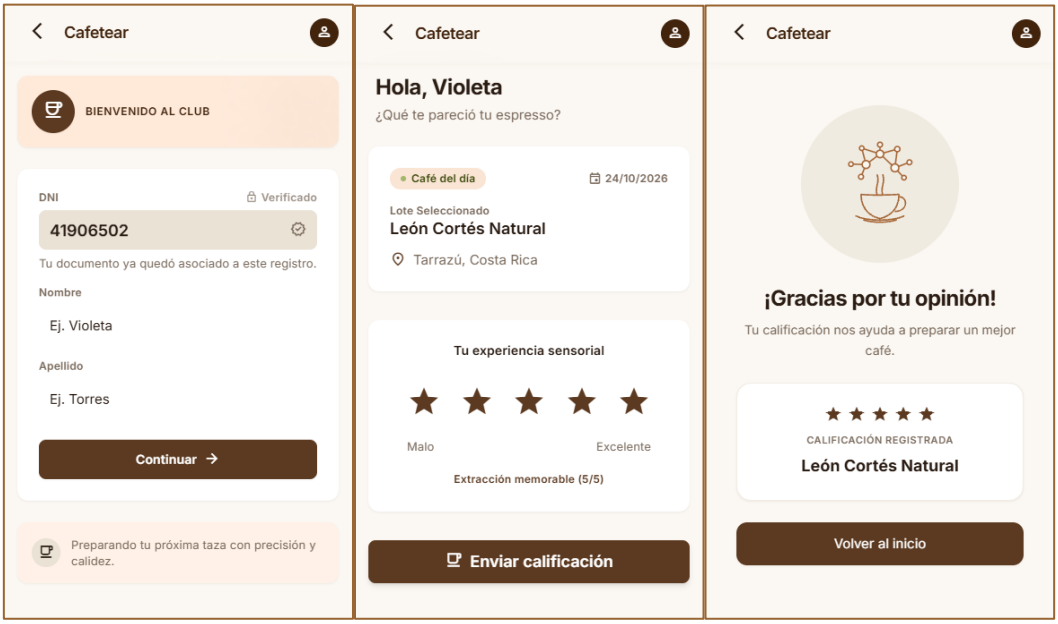
El alcance de este trabajo es la base de datos: no se desarrolló una aplicación. Se diseñaron mockups de cómo se usaría (una aplicación mobile para clientes y una web de uso interno, al final de esta sección). La interacción prevista es mediante procedimientos, de modo que la aplicación solo tenga que llamar a `EXEC`. Ejemplo del flujo de un barista:

```
EXEC uspCalibracionCrear @fechaCalibracion = '2026-11-05T08:30:00', @dosis = 18.0,
                         @tiempoExtraccion = 28, @rendimiento = 36.0, @idBarista =
1, @idLote = 3;
EXEC uspCalificacionCrear @fecha = '2026-11-05T10:00:00', @puntaje = 5, @idCliente =
1;
SELECT * FROM vwMejorCalibracionPorGrano;       -- la receta de referencia de cada
grano
```

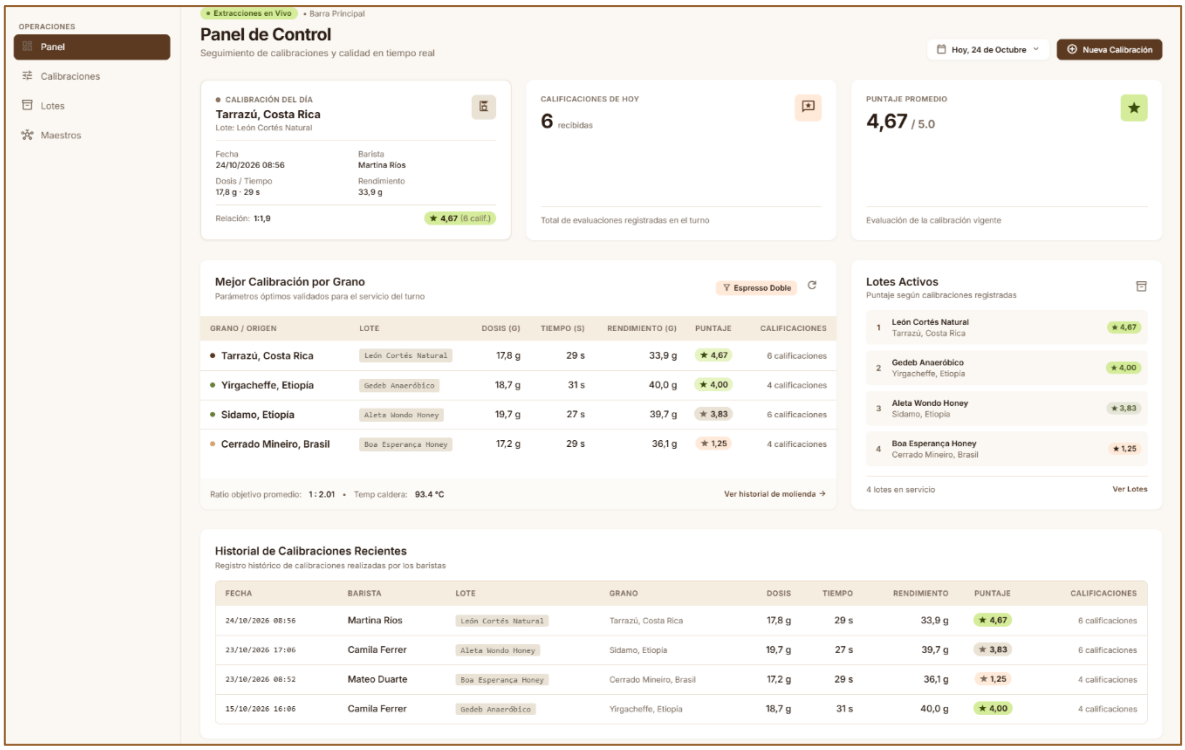
Aplicación Mobile para clientes:

Fecha

Trabajo Practico Integrador Obligatorio



Aplicación web de uso interno:



8. Plan de Desarrollo y Metodología

Se trabajó con **SCRUM** en tres sprints cortos, con revisión cruzada entre los integrantes de lo que cada uno desarrollaba.

Sprint	Objetivo	Entregables
1	Entender el dominio y diseñar	Propuesta, DER, modelo lógico, diccionario de datos
2	Construir	Tablas, datos de prueba y verificación de la carga
3	Cerrar y validar	Vistas, funciones, triggers, índices, seguridad, backup, pruebas e informe

Roles: Product Owner (definición del negocio y prioridades), Scrum Master (coordinación) y equipo de desarrollo (los seis integrantes, que escriben y revisan consultas). Herramientas: SQL Server y SSMS.

9. Optimización y Rendimiento

SQL Server no crea índices automáticamente sobre las claves foráneas. Se crearon tres, sobre las columnas que más se usan en `JOIN` y filtros:

indice	tabla	columna
ixCalibracionFecha	Calibracion	fechaCalibracion
ixCalibracionIdLote	Calibracion	idLote
ixCalificacionIdCalibracion	Calificacion	idCalibracion

Medición. Con `SET STATISTICS IO ON` se comparó una búsqueda puntual (`WHERE idCalibracion = 100` sobre `Calificacion`) con y sin el índice: con índice leyó 2 páginas y sin él 4, porque pasa de recorrer toda la tabla a ir directo al dato. La diferencia es mínima, y es lo esperable: con 346 calificaciones la tabla ocupa 4 páginas. En una consulta de promedios por lote que recorre casi todas las filas, los índices incluso no ayudan (el optimizador prefiere recorrer la tabla completa). Su beneficio real aparece cuando el historial crece (más sucursales y años de calibraciones); se dejaron creados porque el costo es mínimo y el trigger de asignación busca por fecha de calibración.

10.Seguridad y Recuperación

10.1.Roles y permisos

Rol	Puede	No puede
rolBarista	Ejecutar `uspCalibracionCrear` y `uspCalificacionCrear`, y leer la vista `vwMejorCalibracionPorGrano`	Leer ni modificar tablas directamente; ver el ranking de baristas
rolDuenio	Leer todo (tablas y vistas) y ejecutar todos los procedimientos, incluido `uspRankingBaristas`	—

El barista no tiene permiso sobre las tablas: registra datos solo mediante el procedimiento (encadenamiento de propiedad). Prueba real con un usuario de prueba en `rolBarista`:

```
OK: el barista puede registrar una calibración con el procedimiento, sin permiso
sobre la tabla
OK: el barista puede registrar una calificación con el procedimiento
OK: el barista puede leer la mejor calibración por grano
OK: The SELECT permission was denied on the object 'Calificacion', database
'cafetear', schema 'dbo'.
OK: The EXECUTE permission was denied on the object 'uspRankingBaristas', database
'cafetear', schema 'dbo'.
OK: el dueño puede leer las tablas y ver el ranking de baristas
```

10.2.Respaldo y recuperación

Se genera una copia completa con suma de verificación y luego se valida que se pueda restaurar:

```
-- 1) Copia completa, con control de integridad.
BACKUP DATABASE cafetear
TO DISK = N'cafetear.bak'
WITH INIT, CHECKSUM, STATS = 25;
GO

-- 2) Verificar que la copia se puede restaurar (no toca la base).
RESTORE VERIFYONLY
```

Resultado medido: 762 páginas respaldadas en 0,155 segundos y `RESTORE VERIFYONLY` informó que el respaldo es válido. La restauración completa queda indicada como comentario, porque pisa la base.

11.Pruebas y Validación

Se probaron la función, las vistas, los procedimientos, los triggers y los permisos, dejando la base como estaba. Cada caso de error debe decir `OK`. Resultados reales, incluidos los que pide la consigna (texto en un número, tamaño excedido, valores fuera de rango):

Caso	Resultado obtenido
Insertar texto ('mucha') en la experiencia de un barista, que es un número	Conversion failed when converting the varchar value 'mucha' to data type int.
DNI de 12 caracteres en una columna de 8	String or binary data would be truncated in table 'cafetear.dbo.Cliente', column 'dni'.
Cliente sin nombre (NULL)	Cannot insert the value NULL into column 'nombre' … column does not allow nulls.
Cliente con un DNI repetido	Violation of UNIQUE KEY constraint … Cannot insert duplicate key in object 'dbo.Cliente'.
Calibración con un lote que no existe	The INSERT statement conflicted with the FOREIGN KEY constraint … table "dbo.Lote"
Calibración con dosis = 0 (procedimiento)	La dosis debe ser mayor a cero.
Calibración anterior al ingreso del lote (procedimiento)	La calibración no puede ser anterior al ingreso del lote.
Calificación con puntaje 9 (trigger)	El puntaje debe estar entre 1 y 5.
Calificación anterior a la calibración (trigger)	La calificación no puede ser anterior a la calibración.
Mismo cliente califica dos veces la misma calibración (trigger)	Ese cliente ya calificó esta calibración.
Calificación en una fecha sin calibración anterior (trigger)	No hay ninguna calibración anterior a esa fecha.
Calificación sin indicar la calibración (trigger INSTEAD OF)	OK: el trigger asignó la calibración más próxima (226)
Estado final de la base	OK: la base quedó igual que antes

12.Gestión de Riesgos

Riesgo	Probabilidad	Impacto	Mitigación
Que cada integrante tenga datos distintos	Alta	Medio	Datos de prueba fijos y verificación de la carga al terminar
Datos cargados con errores	Media	Alto	Triggers, claves y procedimientos con validación
Pérdida de datos	Baja	Alto	Backup con checksum y `RESTORE VERIFYONLY`
Acceso indebido a las tablas	Media	Alto	Roles y permisos solo por procedimientos
Crecimiento del volumen de datos	Media	Medio	Índices sobre claves foráneas y fecha
Cambios de alcance (por ejemplo mezclas de granos)	Media	Medio	Se dejó fuera de alcance; el modelo admite agregar una tabla Receta

13.Costos y Presupuesto

Estimación académica de recursos humanos, hardware y software. Los valores son de referencia y los debe confirmar el grupo.

Recursos humanos (6 integrantes, a USD 8 por hora)

Actividad	Horas por integrante	Horas totales	Costo (USD)
Diseño y modelado (propuesta, DER, diccionario de datos)	30	180	1.440
Desarrollo y pruebas (datos, procedimientos, triggers, consultas)	40	240	1.920
Documentación e informe	10	60	480
Total	**80**	**480**	**3.840**

Hardware y software

Recurso	Detalle	Costo (USD)
Equipos de desarrollo	Notebook propia de cada integrante, con 8 GB de RAM o más para correr SQL Server	0
Servidor de pruebas	SQL Server instalado en cada equipo (Express o Developer): no hace falta un servidor dedicado	0
Software de desarrollo	SQL Server 2022 Developer (gratuita, solo para desarrollo y pruebas) y SSMS (gratuito)	0
Servidor de producción (si el sistema se pone en marcha)	Máquina virtual de 4 núcleos, 16 GB de RAM y 100 GB de disco SSD. La base de datos ocupa muy poco: el respaldo actual pesa unos 6 MB	A cotizar (mensual)
Licencia de producción	SQL Server Express es gratuita y admite bases de hasta 10 GB, más que suficiente para este volumen; Standard queda como alternativa si crece	0 a cotizar

Total estimado del proyecto académico: USD 3.840, que corresponden solo a las horas de trabajo. El hardware y las licencias de desarrollo no tienen costo. Los costos de una puesta en producción (servidor y eventual licencia) quedan a cotizar.

14.Conclusiones y Recomendaciones

Cafetear convierte la calibración, que hoy es intuición, en información consultable: permite saber qué receta funciona mejor con cada grano, ver si la calidad sube o baja mes a mes y detectar lotes o prácticas que dañan la experiencia del cliente. El modelo es simple a propósito (siete tablas, sin relaciones muchos-a-muchos), y las reglas importantes viven en la base, no en la buena voluntad de quien carga los datos.

Recomendaciones y próximos pasos:

- Agregar una interfaz (web o app) que llame a los procedimientos desde el puesto del barista.
- Incorporar una tabla de sucursales para comparar locales entre sí.
- Evaluar mezclas de granos (tabla Receta) si el negocio las vuelve habituales.
- Registrar variables que hoy no se miden (temperatura, humedad, molienda) para explicar mejor las diferencias de calidad.

- Incorporar funciones de ventana (`ROW_NUMBER`, `LAG`) cuando se vean en la materia, para tendencias más finas.

Limitaciones: los datos son de prueba generados para simular un año de operación; las conclusiones numéricas ilustran el funcionamiento del sistema y no representan una cafetería real.

15.Referencias y Anexos

- Microsoft Learn: T-SQL (https://learn.microsoft.com/sql/t-sql) – `CREATE TRIGGER`, `CREATE VIEW`, `CREATE PROCEDURE`, `THROW`, `BACKUP DATABASE`.
- Material de la cátedra: clases 6 y 7 (vistas, subconsultas, triggers, procedimientos y funciones, índices) – Ing. Franco Emanuel Salazar.